

12 — Graphs Advanced

DSA in Python — Interview Prep

Graphs Advanced — Tutorial

Covers the 10 problems solved in this folder: Union-Find and three applications, then the shortest-path family (Dijkstra, a minimax variant, Bellman-Ford, Floyd-Warshall), then Minimum Spanning Trees via both Kruskal's and Prim's. Where `11_graphs` was almost entirely BFS/DFS, this topic is about **weighted** graphs and the specific algorithm each weight-related question calls for.

The patterns

1. Union-Find as a foundation (01)

Every node starts as its own set; `find(x)` walks to the set's root with path compression (repointing visited nodes directly to the root, so future lookups are near $O(1)$); `union(x, y)` merges two sets by attaching the smaller-rank tree under the larger one. The single most useful fact about it: `union()`'s return value (whether the two nodes were already connected) *is* a cycle/redundancy check, for free — no separate DFS needed.

2. Union-Find for explicit-edge cycle/component problems (02, 03)

- `redundant_connection`: process edges in order, union each pair; the first edge whose endpoints are already connected is the one closing a cycle.
- `number_of_provinces`: union every directly-connected pair from an adjacency matrix; the number of distinct roots left is the number of components (the same question `11_graphs/06` solved via DFS — worth comparing both approaches side by side to build the instinct for recognizing this problem shape either way).

3. Union-Find over derived relationships, not explicit edges (04)

`accounts_merge` doesn't start with a graph at all — the "edges" are inferred from shared emails between account entries. This generalizes the pattern: whenever records need merging based on **any** shared key (not just an explicit adjacency list), Union-Find over record indices, with unions triggered by shared-key collisions, is the tool.

4. Dijkstra — single-source shortest path, non-negative weights (05)

A min-heap always pops the currently-closest unfinalized node, and because weights are non-negative, that popped distance is guaranteed final the moment it's popped — no cheaper path could ever be found through a node that's farther away. The `if d > dist[node]: continue` guard discards stale heap entries left over from before a better path was found. This guarantee is exactly what breaks with negative edges (a "farther" node could still lead to a big negative-weight shortcut later), which is why Dijkstra requires non-negative weights and why `07` exists for the case where that assumption doesn't hold.

5. Turning Dijkstra into a minimax shortest path (06)

`path_with_minimum_effort` keeps every part of Dijkstra's structure except one line: the relaxation formula changes from `current + weight` (accumulate) to `max(current, weight)` (worst step wins). Recognizing that a

single formula swap adapts Dijkstra to "minimize the worst edge" instead of "minimize the total" is a reusable trick, not a new algorithm to memorize separately.

6. Bellman-Ford — shortest path bounded by edge count (07)

Dijkstra finds the *unconstrained* cheapest path, which may use more edges than allowed. Bellman-Ford instead relaxes *every* edge for exactly $k+1$ rounds, where round i finds the best cost reachable using at most i edges — directly modeling an edge-count limit that Dijkstra has no way to express. The critical implementation detail: each round must relax against a *snapshot* of the previous round's distances, never the array being mutated live, or a single round could silently chain multiple relaxations together and exceed the edge budget it's supposed to represent.

7. Floyd-Warshall — all-pairs shortest path (10)

When shortest paths are needed *between every pair* of nodes (not just from one source), Floyd-Warshall computes all of them in one $O(n^3)$ pass: for every candidate intermediate city k , check whether routing $i \rightarrow k \rightarrow j$ beats the current known $i \rightarrow j$ distance, over the whole matrix. This trades a higher complexity bound for getting every pair's answer in a single unified pass instead of running a single-source algorithm (like 05) n separate times.

8. Kruskal's MST — cheapest edges first, via Union-Find (08)

Sort all edges by weight, then greedily accept any edge connecting two currently-separate components (`union()` returning True is exactly "this edge is useful"); reject any edge that would just create a cycle (`union()` returning False). This is Union-Find's other classic use, distinct from the connectivity-check role in 02-04 — here it's the mechanism that prevents a spanning tree from ever containing a cycle while it's being built greedily.

9. Prim's MST — grow one tree outward, via a heap (09)

When the graph is *complete* (every pair of nodes is a potential edge, as with points on a plane), listing and sorting all $O(n^2)$ edges up front for Kruskal's is wasteful. Prim's instead grows a single tree from one starting node, repeatedly pulling in the cheapest edge connecting the tree to any node not yet in it — same "pop cheapest from a heap, skip if stale" shape as Dijkstra (05), differing only in whether the heap key is distance-from-source or distance-from-the-growing-tree. Distances to unvisited nodes are computed on demand as each new node joins, rather than needing the full edge list precomputed.

Complexity summary

#	Problem	Time	Extra space
01	union_find	$\sim O(1)$ amortized per op	$O(n)$
02	redundant_connection	$\sim O(n \alpha(n))$	$O(n)$
03	number_of_provinces	$O(n^2)$	$O(n)$
04	accounts_merge	$O(\text{total emails} \cdot \alpha(n))$	$O(\text{total emails})$
05	network_delay_time	$O(E \log V)$	$O(V+E)$
06	path_with_minimum_effort	$O(\text{rows} \cdot \text{cols} \cdot \log(\text{rows} \cdot \text{cols}))$	$O(\text{rows} \cdot \text{cols})$
07	cheapest_flights_within_k_stops	$O(k \cdot E)$	$O(V)$
08	connecting_cities_minimum_cost	$O(E \log E)$	$O(V)$
09	min_cost_to_connect_all_points	$O(n^2 \log n)$	$O(n)$

#	Problem	Time	Extra space
10	find_the_city	$O(n^3)$	$O(n^2)$

($\alpha(n)$ is the inverse Ackermann function — effectively constant for any n that could ever appear in practice, which is the formal justification behind calling Union-Find operations "near $O(1)$ ".)

Common interview follow-ups to be ready for

- **union_find**: "what if you skipped path compression or union by rank?" — either one alone still gives correct results but degrades worst-case find() to $O(n)$ (a long chain); be ready to explain why **both together** are what give the near- $O(1)$ amortized bound.
- **redundant_connection**: "what if the graph were directed?" — plain Union-Find no longer directly applies (it's inherently for undirected connectivity); a directed cycle needs the 3-state DFS approach from [11_graphs/07](#) instead.
- **network_delay_time**: "what if edge weights could be negative?" — Dijkstra's correctness guarantee breaks immediately; that's precisely the scenario [07's](#) Bellman-Ford handles instead.
- **path_with_minimum_effort**: "can you solve this with binary search instead?" — yes: binary search on the answer (the max allowed effort), with a BFS/DFS feasibility check at each candidate — a nice alternative to mention, connecting back to [06_binary_search's](#) "binary search on the answer space" pattern.
- **cheapest_flights_within_k_stops**: "why not just use Dijkstra with a stop-count added to the state?" — actually a valid alternative (track (node, stops_used) as the Dijkstra state instead of just node); worth knowing as a second correct approach beyond Bellman-Ford.
- **connecting_cities_minimum_cost / min_cost_to_connect_all_points**: "when would you pick Kruskal's over Prim's, or vice versa?" — sparse explicit edge lists favor Kruskal's (sort once, $O(E \log E)$); dense/complete graphs favor Prim's (avoids materializing $O(n^2)$ edges up front).
- **find_the_city**: "why not just run Dijkstra n times instead of Floyd-Warshall?" — n runs of Dijkstra cost $O(n \cdot E \log V)$, which can beat $O(n^3)$ on sparse graphs; Floyd-Warshall wins specifically when the graph is dense or when its simplicity outweighs the asymptotic difference for the given n .

Self-check before moving on

You should be able to, without looking at the code:

- Write Union-Find's **find** (with path compression) and **union** (with union by rank) from memory.
- Explain precisely why Dijkstra requires non-negative weights and what specifically breaks with a negative edge.
- State the one-line relaxation-formula change that turns Dijkstra into the minimax path algorithm used in [06](#).
- Explain why Bellman-Ford's per-round relaxation must use a snapshot of the previous round's distances, with a concrete example of what goes wrong without it.
- State when Kruskal's is preferable to Prim's and vice versa, in one sentence each.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to [13_dynamic_programming_1d](#).

Programs

01. Union Find

01_union_find.py

Explanation

Problem: Implement a Union-Find (Disjoint Set Union) structure supporting `find(x)` (which set does `x` belong to), `union(x, y)` (merge the sets containing `x` and `y`, returning `False` if they were already in the same set - i.e. the edge `(x, y)` would create a cycle), and tracking the current number of disjoint sets.

Source : foundational structure, used throughout this folder (not a single numbered LeetCode problem)

Example:

```
uf = UnionFind(5)
uf.union(0, 1) -> True (merged into one set)
uf.union(1, 2) -> True
uf.connected(0, 2) -> True (both in the same set now)
uf.union(0, 2) -> False (already connected - redundant edge)
uf.count -> 3 ({0,1,2}, {3}, {4})
```

Idea: every node starts as its own parent (its own set). `find(x)` walks up parent pointers until it hits a node that's its own parent (the set's "root" / representative), with **path compression** - every node visited along the way gets repointed directly to the root - so future `find()` calls on those nodes are $O(1)$. `union(x, y)` finds both roots; if they're already equal, `x` and `y` are already connected (this is exactly how 02's redundant-connection detection works - the first edge whose endpoints are already connected is the one creating a cycle). Otherwise, attach the smaller-rank tree under the larger-rank one ("union by rank") to keep trees shallow. Together, path compression and union by rank give near- $O(1)$ amortized `find/union` - the reason Union-Find beats a plain DFS-based connectivity check when there are many union/find queries interleaved.

Code

```
class UnionFind:
    def __init__(self, n: int):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.count = n

    def find(self, x: int) -> int:
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
```

```

def union(self, x: int, y: int) -> bool:
    root_x, root_y = self.find(x), self.find(y)
    if root_x == root_y:
        return False

    if self.rank[root_x] < self.rank[root_y]:
        root_x, root_y = root_y, root_x
    self.parent[root_y] = root_x
    if self.rank[root_x] == self.rank[root_y]:
        self.rank[root_x] += 1

    self.count -= 1
    return True

def connected(self, x: int, y: int) -> bool:
    return self.find(x) == self.find(y)

if __name__ == "__main__":
    uf = UnionFind(5)
    ops = [
        (uf.union, (0, 1), True),
        (uf.union, (1, 2), True),
        (uf.connected, (0, 2), True),
        (uf.union, (0, 2), False),
        (lambda: uf.count, (), 3),
        (uf.union, (3, 4), True),
        (lambda: uf.count, (), 2),
        (uf.connected, (0, 4), False),
    ]

    for i, (fn, args, expected) in enumerate(ops, 1):
        got = fn(*args)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

02. Redundant Connection

02_redundant_connection.py

Explanation

Problem: A tree with n nodes had one extra edge added, making it a graph with exactly one cycle. Given the edges in the order they were added, return the edge that, if removed, restores a valid tree (if several edges could be removed, return the one that appears last in the input).

Source : LeetCode 684 - Redundant Connection

Example:

Input: `[[1,2], [1,3], [2,3]]`

Output: `[2, 3]`

Idea: the first direct application of 01's Union-Find. Process edges in order, union-ing each pair's endpoints. The moment an edge's two endpoints are **already** in the same set (*find* gives the same root for both), that edge doesn't connect anything new - it's the one closing the cycle, and since edges are processed in input order, it's

automatically the last such edge (the answer the problem wants). No separate cycle-detection DFS needed - Union-Find's own union() return value *is* the cycle check.

Code

```
def find_redundant_connection(edges: list[list[int]]) -> list[int]:
    n = len(edges)
    parent = list(range(n + 1)) # nodes are 1-indexed

    def find(x: int) -> int:
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    for a, b in edges:
        root_a, root_b = find(a), find(b)
        if root_a == root_b:
            return [a, b]
        parent[root_a] = root_b

    return []

if __name__ == "__main__":
    tests = [
        ([[1, 2], [1, 3], [2, 3]], [2, 3]),
        ([[1, 2], [2, 3], [3, 4], [1, 4], [1, 5]], [1, 4]),
    ]

    for i, (edges, expected) in enumerate(tests, 1):
        got = find_redundant_connection(edges)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

03. Number Of Provinces

03_number_of_provinces.py

Explanation

Problem: Given an $n \times n$ adjacency matrix `is_connected` where `is_connected[i][j] = 1` means cities i and j are directly connected, return the number of provinces (connected groups of cities).

Source : LeetCode 547 - Number of Provinces

Example:

Input: `[[1,1,0],[1,1,0],[0,0,1]]`

Output: 2

Idea: [11_graphs/06_number_of_connected_components](#) solved this exact question with DFS. Here it's solved with Union-Find instead, to make the comparison concrete: union every directly-connected pair, then the number of *distinct roots* remaining across all nodes is the number of provinces. Both approaches are $O(n^2)$ here (the matrix itself forces an $O(n^2)$ scan), so this problem doesn't demonstrate a

speed advantage for Union-Find - it's here to build the pattern-matching instinct for recognizing "count connected components" as a Union-Find-shaped problem, not just a DFS-shaped one.

Code

```
def find_circle_num(is_connected: list[list[int]]) -> int:
    n = len(is_connected)
    parent = list(range(n))

    def find(x: int) -> int:
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    def union(x: int, y: int) -> None:
        root_x, root_y = find(x), find(y)
        if root_x != root_y:
            parent[root_x] = root_y

    for i in range(n):
        for j in range(i + 1, n):
            if is_connected[i][j] == 1:
                union(i, j)

    return len({find(i) for i in range(n)})

if __name__ == "__main__":
    tests = [
        ([[1, 1, 0], [1, 1, 0], [0, 0, 1]], 2),
        ([[1, 0, 0], [0, 1, 0], [0, 0, 1]], 3),
        ([[1, 1, 1], [1, 1, 1], [1, 1, 1]], 1),
    ]

    for i, (matrix, expected) in enumerate(tests, 1):
        got = find_circle_num(matrix)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

04. Accounts Merge

04_accounts_merge.py

Explanation

Problem: Given a list of accounts, each [name, email1, email2, ...], merge accounts that share at least one email (same person may have listed slightly different names, but shared emails prove it's the same account). Return the merged accounts, each with the name first and all emails sorted, deduplicated.

Source : LeetCode 721 - Accounts Merge

Example:

Input: `[["John", "johnsmith@mail.com", "john_newyork@mail.com"],`
`["John", "johnsmith@mail.com", "john00@mail.com"],`
`["Mary", "mary@mail.com"]]`

```
["John","johnnybravo@mail.com"]]
```

```
Output: [["John","john00@mail.com","john_newyork@mail.com","johnsmith@mail.com"],  
["Mary","mary@mail.com"],  
["John","johnnybravo@mail.com"]]
```

Idea: a less obvious Union-Find application than 02/03 - the union-find here operates over *account indices*, not over an explicit graph of nodes/edges. Walk every account's emails; the first time an email is seen, remember which account index it came from. If that same email shows up again under a different account, union those two account indices together - shared emails are the "edges" proving two account entries belong to the same real person. After processing everything, group emails by their account's *root* index, and each group becomes one merged account. This pattern - deriving union operations from shared attributes rather than an explicit edge list - generalizes to any "merge records that share some key" problem.

Code

```
def accounts_merge(accounts: list[list[str]]) -> list[list[str]]:  
    n = len(accounts)  
    parent = list(range(n))  
  
    def find(x: int) -> int:  
        if parent[x] != x:  
            parent[x] = find(parent[x])  
        return parent[x]  
  
    def union(x: int, y: int) -> None:  
        root_x, root_y = find(x), find(y)  
        if root_x != root_y:  
            parent[root_x] = root_y  
  
    email_to_account: dict[str, int] = {}  
    for i, account in enumerate(accounts):  
        for email in account[1:]:  
            if email in email_to_account:  
                union(i, email_to_account[email])  
            else:  
                email_to_account[email] = i  
  
    grouped: dict[int, set[str]] = {}  
    for email, i in email_to_account.items():  
        root = find(i)  
        grouped.setdefault(root, set()).add(email)  
  
    return [[accounts[root][0]] + sorted(emails) for root, emails in grouped.items()]  
  
def _normalize(merged):  
    return sorted(tuple(acc) for acc in merged)  
  
if __name__ == "__main__":  
    tests = [  
        ([  
            ["John", "johnsmith@mail.com", "john_newyork@mail.com"],  
            ["John", "johnsmith@mail.com", "john00@mail.com"],
```

```

        ["Mary", "mary@mail.com"],
        ["John", "johnnybravo@mail.com"],
    ], [
        ["John", "john00@mail.com", "john_newyork@mail.com", "johnsmith@mail.com"],
        ["Mary", "mary@mail.com"],
        ["John", "johnnybravo@mail.com"],
    ]),
    ([[ "A", "a1@mail.com" ]], [[ "A", "a1@mail.com" ]]),
]

for i, (accounts, expected) in enumerate(tests, 1):
    got = accounts_merge(accounts)
    status = "PASS" if _normalize(got) == _normalize(expected) else "FAIL"
    print(f"Test {i}: {status} (got {got}, expected {expected})")

```

05. Network Delay Time

05_network_delay_time.py

Explanation

Problem: A network has n nodes (1 to n) and directed weighted edges $\text{times} = [[u, v, w], \dots]$ meaning a signal takes w time to travel from u to v . Given a starting node k , return the time for a signal sent from k to reach every node (the time when the *last* node receives it), or -1 if some node is unreachable.

Source : LeetCode 743 - Network Delay Time

Example:

Input: $\text{times} = [[2,1,1],[2,3,1],[3,4,1]]$, $n = 4$, $k = 2$

Output: 2

Idea: the canonical Dijkstra's algorithm - single-source shortest paths on a graph with non-negative weights. A min-heap always pops the currently-closest unfinalized node next, guaranteeing that by the time a node is popped, its **dist** value is already the true shortest distance to it (any other path to it would have to go through a node that's farther away, which can't produce a shorter result with non-negative weights - this is exactly why Dijkstra breaks with negative edges, and why 07's Bellman-Ford exists as the fallback for that case). The **if d > dist[node]: continue** guard skips stale heap entries left over from a node being reached via a worse path earlier, before a better path to it was later found and pushed again.

Code

```

import heapq

def network_delay_time(times: list[list[int]], n: int, k: int) -> int:
    adj: dict[int, list[tuple[int, int]]] = {i: [] for i in range(1, n + 1)}
    for u, v, w in times:
        adj[u].append((v, w))

    dist = {i: float("inf") for i in range(1, n + 1)}
    dist[k] = 0

```

```

heap = [(0, k)]

while heap:
    d, node = heapq.heappop(heap)
    if d > dist[node]:
        continue
    for neighbor, weight in adj[node]:
        new_dist = d + weight
        if new_dist < dist[neighbor]:
            dist[neighbor] = new_dist
            heapq.heappush(heap, (new_dist, neighbor))

max_dist = max(dist.values())
return max_dist if max_dist < float("inf") else -1

if __name__ == "__main__":
    tests = [
        ([[2, 1, 1], [2, 3, 1], [3, 4, 1]], 4, 2, 2),
        ([[1, 2, 1]], 2, 1, 1),
        ([[1, 2, 1]], 2, 2, -1),
    ]

    for i, (times, n, k, expected) in enumerate(tests, 1):
        got = network_delay_time(times, n, k)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

06. Path With Minimum Effort

06_path_with_minimum_effort.py

Explanation

Problem: Given a grid of heights, find a path from the top-left to the bottom-right (4-directional moves) that minimizes the *maximum* absolute height difference between any two consecutive cells along the path. Return that minimized maximum difference.

Source : LeetCode 1631 - Path With Minimum Effort

Example:

Input: `[[1,2,2],[3,8,2],[5,3,5]]`

Output: 2

Idea: a "minimax" variant of Dijkstra - instead of summing edge weights along the path (like 05), the cost of a path is its single *worst* step, and the goal is to minimize that worst step across all possible paths. The algorithm structure barely changes: still a min-heap keyed by the best-known cost to reach each cell, still relaxing neighbors and pushing improvements. The one change is the relaxation formula itself - `new_cost = max(current_cost, abs(height_diff))` instead of `current_cost + weight` - because extending a path by one step doesn't accumulate effort, it only raises the effort if this new step is worse than every step so far. This same swap (replace `+` with `max` in the relaxation) turns Dijkstra into a minimax-shortest-path algorithm for any problem

framed as "minimize the worst edge on the path" instead of "minimize the total path weight."

Code

```
import heapq

def minimum_effort_path(heights: list[list[int]]) -> int:
    rows, cols = len(heights), len(heights[0])
    effort = [[float("inf")] * cols for _ in range(rows)]
    effort[0][0] = 0
    heap = [(0, 0, 0)] # (effort_so_far, row, col)

    while heap:
        e, r, c = heapq.heappop(heap)
        if e > effort[r][c]:
            continue
        if r == rows - 1 and c == cols - 1:
            return e
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                new_effort = max(e, abs(heights[nr][nc] - heights[r][c]))
                if new_effort < effort[nr][nc]:
                    effort[nr][nc] = new_effort
                    heapq.heappush(heap, (new_effort, nr, nc))

    return 0

if __name__ == "__main__":
    tests = [
        ([[1, 2, 2], [3, 8, 2], [5, 3, 5]], 2),
        ([[1, 2, 3], [3, 8, 4], [5, 3, 5]], 1),
        ([[1, 2, 1, 1, 1], [1, 2, 1, 2, 1], [1, 2, 1, 2, 1],
          [1, 2, 1, 2, 1], [1, 1, 1, 2, 1]], 0),
    ]

    for i, (heights, expected) in enumerate(tests, 1):
        got = minimum_effort_path(heights)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

07. Cheapest Flights Within K Stops

07_cheapest_flights_within_k_stops.py

Explanation

Problem: Given n cities, directed weighted flight routes, a source, a destination, and a maximum of k stops (k+1 flights) allowed, return the cheapest price to get from source to destination within that limit, or -1 if impossible.

Source : LeetCode 787 - Cheapest Flights Within K Stops

Example:

Input: n=4, flights=[[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]],

src=0, dst=3, k=1

Output: 700 (0 -> 1 -> 3, exactly 1 stop, cost 100+600)

Idea: Dijkstra (05) always finds the *unconstrained* cheapest path, which could use more stops than allowed here - so it doesn't directly apply. Bellman-Ford instead relaxes *every* edge, for exactly $k+1$ rounds (one round per flight allowed) - round i finds the cheapest cost reachable using at most i flights. The critical detail: each round must relax edges using a *snapshot* of the previous round's distances (`new_dist = dist[:]`, updated separately, then swapped in after the round), not the array being mutated live - otherwise a single round could accidentally chain multiple edge relaxations together, effectively using more flights than that round is allowed to represent. This snapshot-per-round requirement is exactly what caps the path length at $k+1$ edges, which is the whole reason Bellman-Ford (not Dijkstra) is the right tool whenever a shortest-path problem comes with a limit on the number of edges used.

Code

```
def find_cheapest_price(n: int, flights: list[list[int]], src: int, dst: int, k: int) -> int:
    dist = [float("inf")] * n
    dist[src] = 0

    for _ in range(k + 1):
        new_dist = dist[:]
        for u, v, w in flights:
            if dist[u] != float("inf") and dist[u] + w < new_dist[v]:
                new_dist[v] = dist[u] + w
        dist = new_dist

    return dist[dst] if dist[dst] != float("inf") else -1

if __name__ == "__main__":
    tests = [
        (4, [[0, 1, 100], [1, 2, 100], [2, 0, 100], [1, 3, 600], [2, 3, 200]], 0, 3, 1, 700),
        (3, [[0, 1, 100], [1, 2, 100], [0, 2, 500]], 0, 2, 1, 200),
        (3, [[0, 1, 100], [1, 2, 100], [0, 2, 500]], 0, 2, 0, 500),
    ]

    for i, (n, flights, src, dst, k, expected) in enumerate(tests, 1):
        got = find_cheapest_price(n, flights, src, dst, k)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

08. Connecting Cities Minimum Cost

08_connecting_cities_minimum_cost.py

Explanation

Problem: Given n cities (1 to n) and a list of possible connections `[city1, city2, cost]`, return the minimum total cost to connect all cities (directly or indirectly), or -1 if it's impossible.

Source : classic MST problem (LeetCode 1135 - Connecting Cities With Minimum Cost, premium; same shape as Min Cost to Connect All Points, problem 09 in this folder)

Example:

Input: n = 3, connections = [[1,2,5],[1,3,6],[2,3,1]]

Output: 6 (use edges 2-3 (cost 1) and 1-2 (cost 5))

Idea: Kruskal's algorithm - build a Minimum Spanning Tree by considering edges in order from *cheapest to most expensive*, greedily accepting any edge that connects two currently-separate components (rejecting one that would connect two nodes already in the same component, since that would just create a cycle without adding connectivity). This is 01's Union-Find used for its most classic purpose: `union()` returning False is precisely "this edge is redundant," and a spanning tree needs exactly $n-1$ accepted edges - if fewer than that get accepted by the time all edges are considered, some cities are unreachable from others and no valid connection exists. Sorting once up front ($O(E \log E)$) dominates the total cost; every union/find after that is near $O(1)$ amortized.

Code

```
def min_cost_connect_cities(n: int, connections: list[list[int]]) -> int:
    parent = list(range(n + 1)) # cities are 1-indexed

    def find(x: int) -> int:
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    connections = sorted(connections, key=lambda edge: edge[2])
    total_cost = 0
    edges_used = 0

    for u, v, cost in connections:
        root_u, root_v = find(u), find(v)
        if root_u != root_v:
            parent[root_u] = root_v
            total_cost += cost
            edges_used += 1
            if edges_used == n - 1:
                break

    return total_cost if edges_used == n - 1 else -1

if __name__ == "__main__":
    tests = [
        (3, [[1, 2, 5], [1, 3, 6], [2, 3, 1]], 6),
        (4, [[1, 2, 3], [3, 4, 4]], -1),
        (1, [], 0),
    ]

    for i, (n, connections, expected) in enumerate(tests, 1):
        got = min_cost_connect_cities(n, connections)
        status = "PASS" if got == expected else "FAIL"
```

```
print(f"Test {i}: {status} (got {got}, expected {expected})")
```

09. Min Cost To Connect All Points

09_min_cost_to_connect_all_points.py

Explanation

Problem: Given n points on a 2D plane, the cost to connect two points is their Manhattan distance. Return the minimum total cost to connect all points into one network (every pair of points is implicitly a possible edge - a complete graph).

Source : LeetCode 1584 - Min Cost to Connect All Points

Example:

Input: `[[0,0],[2,2],[3,10],[5,2],[7,0]]`

Output: 20

Idea: Prim's algorithm - the other classic way to build an MST, better suited than Kruskal's (08) here specifically because the graph is *complete* (every pair of points is a potential edge, so listing all edges up front would be $O(n^2)$ edges to sort). Prim's instead grows a single tree outward one node at a time: start from any point, and repeatedly add the cheapest edge connecting the tree-so-far to any point not yet in it, tracked with a min-heap (same "pop cheapest, skip if stale/already handled" shape as Dijkstra in 05 - Prim's and Dijkstra are close cousins, differing only in whether the heap key is "distance from the source" or "distance from the growing tree"). Distances to unvisited points are computed on demand as each new point joins the tree, rather than requiring the full edge list upfront - the reason this fits a complete graph better than Kruskal's.

Code

```
import heapq

def min_cost_connect_points(points: list[list[int]]) -> int:
    n = len(points)
    visited = [False] * n
    min_heap = [(0, 0)] # (cost, point_index)
    total_cost = 0
    edges_used = 0

    while edges_used < n:
        cost, i = heapq.heappop(min_heap)
        if visited[i]:
            continue
        visited[i] = True
        total_cost += cost
        edges_used += 1

        xi, yi = points[i]
        for j in range(n):
            if not visited[j]:
                xj, yj = points[j]
```

```

        dist = abs(xi - xj) + abs(yi - yj)
        heapq.heappush(min_heap, (dist, j))

    return total_cost

if __name__ == "__main__":
    tests = [
        ([[0, 0], [2, 2], [3, 10], [5, 2], [7, 0]], 20),
        ([[3, 12], [-2, 5], [-4, 1]], 18),
        ([[0, 0]], 0),
    ]

    for i, (points, expected) in enumerate(tests, 1):
        got = min_cost_connect_points(points)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

10. Find The City

10_find_the_city.py

Explanation

Problem: Given n cities and weighted undirected edges, find the city that has the smallest number of *other* cities reachable within `distance_threshold`. If tied, return the city with the largest index.

Source : LeetCode 1334 - Find the City With the Smallest Number of Neighbors at a Threshold Distance

Example:

Input: $n=4$, `edges=[[0,1,3],[1,2,1],[1,3,4],[2,3,1]]`,

`distance_threshold=4`

Output: 3

Idea: 05's Dijkstra and 07's Bellman-Ford both find shortest paths *from a single source*. This problem needs shortest paths *between every pair* of cities (to count, for each city, how many others are reachable) - running Dijkstra n separate times (once per city) would work, but Floyd-Warshall solves all pairs at once with a different approach: a `dist[i][j]` matrix, initialized directly from the edges, then repeatedly improved by asking "does routing through city k make i to j shorter?" (`dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])`) for every possible intermediate city k , one k at a time, over the *entire* matrix. After considering all n candidate intermediate cities, `dist[i][j]` is guaranteed to be the true shortest path, since any shortest path's full sequence of intermediate cities will have been "discovered" by the time every single city has been tried as an intermediate hop. The $O(n^3)$ cost is the trade-off for getting every pair's answer in one pass instead of n separate single-source runs.

Code

```
def find_the_city(n: int, edges: list[list[int]], distance_threshold: int) -> int:
```

```

INF = float("inf")
dist = [[INF] * n for _ in range(n)]
for i in range(n):
    dist[i][i] = 0
for u, v, w in edges:
    dist[u][v] = min(dist[u][v], w)
    dist[v][u] = min(dist[v][u], w)

for k in range(n):
    for i in range(n):
        for j in range(n):
            if dist[i][k] + dist[k][j] < dist[i][j]:
                dist[i][j] = dist[i][k] + dist[k][j]

best_city = -1
best_count = float("inf")
for i in range(n):
    count = sum(1 for j in range(n) if i != j and dist[i][j] <= distance_threshold)
    if count <= best_count:
        best_count = count
        best_city = i

return best_city

if __name__ == "__main__":
    tests = [
        (4, [[0, 1, 3], [1, 2, 1], [1, 3, 4], [2, 3, 1]], 4, 3),
        (5, [[0, 1, 2], [0, 4, 8], [1, 2, 3], [1, 4, 2], [2, 3, 1], [3, 4, 1]], 2, 0),
    ]

    for i, (n, edges, threshold, expected) in enumerate(tests, 1):
        got = find_the_city(n, edges, threshold)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```